1 Natural Deduction

1.1 From axioms to rules

1.1.1 Reading (pair) backwards

The schema (pair) is an axiom, and an axiom is a one-line proof, so $\Gamma \vdash \varphi \to (\psi \to \varphi \wedge \psi)$ for every Γ at all. Recall the Deduction Theorem: $\Gamma, \varphi \vdash \chi$ exactly when $\Gamma \vdash \varphi \to \chi$. Apply it twice:

$$\begin{aligned} & \Gamma \vdash \varphi \to (\psi \to \varphi \wedge \psi), \\ \text{so} \quad & \Gamma, \varphi \vdash \psi \to \varphi \wedge \psi. \\ \text{so} \quad & \Gamma, \varphi, \psi \vdash \varphi \wedge \psi. \end{aligned}$$

The arrows have come out of the formula and gone above the line, among the hypotheses. Γ, φ is short for $\Gamma \cup \{\varphi\}$.

1.1.2 Splice in the proofs

Now suppose Baker has proved both halves, $\Gamma \vdash \varphi$ and $\Gamma \vdash \psi$. Transitivity says a hypothesis you can prove may be replaced by its proof, so she splices each proof in for its hypothesis. What is left is:

$$\text{if } \Gamma \vdash \varphi \text{ and } \Gamma \vdash \psi, \text{ then } \Gamma \vdash \varphi \wedge \psi$$

The axiom has become a rule: something you do rather than something we're told.

Proposition 1.1 (derived rules)**.** For all sets of formula Γ and all formulas φ, ψ, χ in the Hilbert system:

1. if $\varphi \in \Gamma$ then $\Gamma \vdash \varphi$ (reflexivity);
2. if $\Gamma \vdash \varphi \to \psi$ and $\Gamma \vdash \varphi$ then $\Gamma \vdash \psi$ (modus ponens);
3. if $\Gamma, \varphi \vdash \psi$ then $\Gamma \vdash \varphi \to \psi$ (the Deduction Theorem);
4. if $\Gamma \vdash \varphi$ and $\Gamma \vdash \psi$ then $\Gamma \vdash \varphi \wedge \psi$ (from (pair));
5. if $\Gamma \vdash \varphi \wedge \psi$ then $\Gamma \vdash \varphi$ (from (fst));
6. if $\Gamma \vdash \varphi \wedge \psi$ then $\Gamma \vdash \psi$ (from (snd));
7. if $\Gamma \vdash \varphi$ then $\Gamma \vdash \varphi \vee \psi$; (from (left));
8. if $\Gamma \vdash \psi$ then $\Gamma \vdash \varphi \vee \psi$; (from (right);
9. if $\Gamma \varphi \vee \psi$, and $\Gamma, \varphi \vdash \chi$, and $\Gamma, \psi \vdash \chi$, then $\Gamma \vdash \chi$ (from (absurd));
10. if $\Gamma \vdash \varphi$ then $\Gamma \vdash \psi \to \varphi$; and if $\Gamma \vdash \varphi \to \psi \to \chi$ and $\Gamma \vdash \varphi \to \psi$ then $\Gamma \vdash \varphi \to \chi$. (from (I), (II))

1.1.3 Two facts that discharge

IN two of the facts, a hypothesis appears in the condition and is gone from the conclusion.

$$\text{if } \Gamma, \varphi \vdash \psi \text{ then } \Gamma \vdash \varphi \to \psi$$

$$\text{if } \Gamma \vdash \varphi \vee \psi, \text{ and } \Gamma, \varphi \vdash \chi, \text{ and } \Gamma, \psi \vdash \chi, \text{ then } \Gamma \vdash \chi$$

1.1.4 A fact, drawn as a bar

The fact, about the Hilbert System:

$$\frac{\Gamma \vdash \varphi \qquad \Gamma \vdash \psi}{\Gamma \vdash \varphi \wedge \psi}$$

Gentzen's Bar:

$$\frac{\varphi \qquad \psi}{\varphi \wedge \psi}\,\wedge\mathrm{I}$$

Premises above, conclusion below, the rule on the right. The Γ is not written, it is whatever sits at the leaves.

- A proof is a **tree**. Each bar is one of the facts, drawn as a rule.
- A leaf is **open** until a bar below it pays for it, and only two kinds of bar can: ($\to$ I) and ($\wedge$E).
- What the tree proves is its **root**, *from* its open leaves.

1.2 Derivations as trees

Definition 1.2 (derivations)**.** The derivations are defined inductively, and with each derivation $\mathcal{D}$ we define its conclusion, the formula at the root, and its set of open hypotheses $\mathsf{OH}(\mathcal{D})$, a finite set of formulas.

1. (hypothesis) For every formula φ, the one-node tree φ is a derivation, with conclusion φ and $\mathsf{OH} = \{\varphi\}$
2. ($\to$ I) If $\mathcal{D}$ has conclusion ψ, then for every formula φ,

$$\frac{\begin{array}{c}[\varphi]\\ \mathcal{D}\\ \psi\end{array}}{\varphi \to \psi}\,{\to}\mathrm{I} \qquad \text{is a derivation,} \qquad \mathsf{OH} = \mathsf{OH}(\mathcal{D}) \setminus \{\varphi\}.$$

3. ($\to$E) If $\mathcal{D}_1$ has conclusion $\varphi \to \psi$ and $\mathcal{D}_2$ has conclusion φ, then

$$\frac{\begin{array}{cc}\mathcal{D}_1 & \mathcal{D}_2\\ \varphi \to \psi & \varphi\end{array}}{\psi}\,{\to}\mathrm{E} \qquad \text{is a derivation,} \qquad \mathsf{OH} = \mathsf{OH}(\mathcal{D}_1) \cup \mathsf{OH}(\mathcal{D}_2).$$

4. ($\wedge$I) If $\mathcal{D}_1$ concludes φ and $\mathcal{D}_2$ concludes ψ, then

$$\frac{\begin{array}{cc}\mathcal{D}_1 & \mathcal{D}_2\\ \varphi & \psi\end{array}}{\varphi \wedge \psi}\,\wedge\mathrm{I} \qquad \text{is a derivation,} \qquad \mathsf{OH} = \mathsf{OH}(\mathcal{D}_1) \cup \mathsf{OH}(\mathcal{D}_2).$$

5. ($\wedge$E) If $\mathcal{D}$ concludes $\varphi \wedge \psi$, then

$$\frac{\begin{array}{c}\mathcal{D}\\ \varphi \wedge \psi\end{array}}{\varphi}\,\wedge\mathsf{E}_1 \qquad \text{and} \qquad \frac{\begin{array}{c}\mathcal{D}\\ \varphi \wedge \psi\end{array}}{\psi}\,\wedge\mathsf{E}_2$$

6. ($\vee$I) If $\mathcal{D}$ concludes φ, then for every formula ψ the tree with root $\varphi \vee \psi$ and immediate subtree $\mathcal{D}$ is a derivation, labelled $\vee I_1$; symmetrically from a derivation of ψ, labelled $\vee I_2$. In both cases $\mathsf{OH} = \mathsf{OH}(\mathcal{D})$.
7. ($\vee$E) If $\mathcal{D}_0$ concludes $\varphi \vee \psi$, and $\mathcal{D}_1$ and $\mathcal{D}_2$ both conclude χ, then

$$\dfrac{\begin{array}{ccc} & [\varphi] & [\psi] \\ \mathcal{D}_0 & \mathcal{D}_1 & \mathcal{D}_2 \\ \varphi \vee \psi & \chi & \chi \end{array}}{\chi} \vee\mathsf{E}$$

8. ($\bot$E) If $\mathcal{D}$ concludes $\bot$, then for every formula χ the tree with root χ and immediate subtree $\mathcal{D}$ is a derivation, with $\mathsf{OH} = \mathsf{OH}(\mathcal{D})$.
9. Nothing else is a derivation.

We write $\Gamma \vdash_N \varphi$, and say that φ is *derivable from* Γ when **some** derivation has conclusion φ and all its open hypotheses in Γ: $\mathsf{OH} \subseteq \Gamma$. Such a derivation witnesses $\Gamma \vdash_N \varphi$. Negation is an abbreviation as before: $\neg\varphi$ is $\varphi \to \bot$, and has no rules of its own.

1.3 Equivalence with the Hilbert system

Theorem 1.3 (equivalence with the Hilbert System)**.** We can now state the theorem promised at the start of the chapter. NJ has no axioms and nine rules; the Hilbert system has nine schemas and one rule. They prove the same things.
For every set of formulas Γ and every formula φ built from letters and $\bot$ with $\to, \wedge$ and $\vee$:

$$\Gamma \vdash_N \varphi \text{ if and only if } \Gamma \vdash \varphi$$

The slides contain the proofs. Reference them if required.

1.4 The λ-calculus

What we had: A Hilbert proof is a typed combinator program with the program rubbed out: (I) is **K**, (II) is **S**, a hypothesis is a variable, modus ponens is application.
What has changed: We have changed what a proof is: a tree, with suppositions that are made and then discharged. So what is the matching notion of a **typed program**?

1.4.1 The Deduciton Theorem, as a program

The new proofs have one move the Hilbert system lacks as a rule: discharge, the Deduction Theorem. The new programs need a term constructor for it.
The Deduction Theorem:

$$\dfrac{\Gamma,\, A \vdash B}{\Gamma \vdash A \to B}$$

With the programs written on:

$$\dfrac{\Gamma, x : A \vdash M : B}{\Gamma, \vdash \lambda x.M : A \to B}$$

Above the line, a program M with a parameter $x : A$; below, the function `\x -> M`.

Definition 1.4 (λ-terms)**.** Fix a stock of variables $x, y, z, f, \ldots$. The terms are defined inductively:

1. every variable x is a term;
2. if M and N are terms then $M\,N$ is a term, the application of M to N;
3. if x is a variable and m is a term then $\lambda x.M$ is a term, the abstraction of M over x: the function that takes x and returns M;
4. nothing else is a term.

In one line:

$$M, N ::= \quad x \quad | \quad M\,N \quad | \quad \lambda x.M$$

Application groups to the left. A λ reaches as far right as it can: $\lambda x.M\,N$ is $\lambda x.(M\,N)$.

1.4.2 Free variables

The λ in $\lambda x.M$ *binds* the parameter x; its *scope* is the body M. An occurrence of a variable is bound if it lies in the scope of a λ on that variable, and free otherwise. A term with no free variables is closed: a whole program. A bound variable may be renamed at will.

1.4.3 Substitution

$M(N/x)$ replaces the free occurrences of x in M by N, and nothing else. With binders about, that takes care.

1. $x(N/x) = N$
2. $y(N/x) = y$, for y other than x
3. $(M_1 M_2)(N/x) = M_1(N/x)M_2(N/x)$
4. $(\lambda x.M_1)(N/x) = \lambda x.M_1$ (every x inside is bound)
5. $(\lambda y.M_1)(N/x) = \lambda y.M_1(N/x)$, **provided** $y \notin \mathsf{FV}(N)$

The side condition on clause 5 is the one subtlety. Pushing N under a λy when y is free in N would capture that y and change what N means. Rename it first.

Example 1.5 (capture)**.** Compute $(\lambda y.x\,y)(y/x)$: the function that applies x to its argument, with y put for x. The answer ought to be the function that applies y to its argument.

Blindly: $\lambda y.y\,y$; the free y has been caught by the λy: a function that applies its argument to itself. Incorrect.

Renaming first: $(\lambda z.x\,z)(y/z) = \lambda z.y\,z$; The function that applies y to its argument. Correct.

1.4.4 Reduction

There is one rule of computation. To apply a function to an argument, substitute the argument for the parameter, anywhere inside a term:

$$(\lambda x.M)N \rightsquigarrow M(N/x)$$

The pair on the left is a redex, and the step is called β. That's what a function call does, and it's all a function call does. A term with no redex is in normal form.

Theorem 1.6 (the Church-Rosser theorem (again))**.** If $M \rightsquigarrow^* N_1$ and $M \rightsquigarrow^* N_2$, then there is a term P such that $N_1 \rightsquigarrow^* P$ and $N_2 \rightsquigarrow^* P$. So, a term has at most one normal form.

Example 1.7 (running terms)**.** Not every terms reaches a normal form: nothing in the definition says what may be applied to what.
A numeral, applied:

$$\begin{aligned}&(\lambda f.\lambda x.f(f\,x))g\,a\\ \rightsquigarrow&(\lambda x.g(g\,x))a\\ \rightsquigarrow&g(g\,a)\end{aligned}$$

A term that never stops: With ω for $\lambda x.x\,x$:

$$\begin{aligned}\Omega &= \omega\omega\\ &\rightsquigarrow (x\,x)(\omega/x) = \omega\omega\\ &\rightsquigarrow \omega\omega \rightsquigarrow \dots\end{aligned}$$

1.4.5 The combinators are terms

$$\mathbf{K} := \lambda x.\lambda y.x$$

$\mathbf{K}\,M\,N \rightsquigarrow M$ is two β steps: $(\lambda x.\lambda y.\,x)M\,N \rightsquigarrow (\lambda y.\,M)N \rightsquigarrow M$

$$\mathbf{S} := \lambda x.\lambda y.\lambda z.\,x\,z(y\,z)$$

$\mathbf{S}\,M\,N\,P \rightsquigarrow M\,P(N\,P)$ is three β steps: one for each parameter. So every combinator term is a λ-term, and the two constants need no longer be taken as given.

Theorem 1.8 (translation into combinators)**.** Every λ-term M translates to a term M° built from $\mathbf{K}$, $\mathbf{S}$, variables and applications, with no λ in it, such that M° has the same free variables as M and does the same thing to every argument: if M is $\lambda x.P$ then $M^\circ N \rightsquigarrow^* P^\circ(N/x)$ for every N.
Conversely, every combinator term is a λ-term.
The translation: translate the innermost abstractions first. For a term P containing no λ, define $[x]P$, "P abstracted over x", by recursion on P:

$$\begin{aligned}[x]P &= \mathsf{K}\,P && \text{if } x \text{ does not occur in } P,\\ [x]x &= \mathsf{S}\,\mathsf{K}\,\mathsf{K}, &&\\ [x](P_1\,P_2) &= \mathsf{S}\,([x]P_1)\,([x]P_2) && \text{otherwise.}\end{aligned}$$

Then, $[x]P$ contains no λ and no x, and $([x]P)N \rightsquigarrow^* P(N/x)$ for every term N. Then replace an innermost $\lambda x.\,P$ by $[x]P$, and repeat until no λ is left.

Example 1.9 (compiling $\lambda x.\,f\,x$)**.** $f\,x$ is an application in which x occurs, so the third clause applies; then the first clause for f, the second for x. Write $\mathbf{I}$ for $\mathbf{S}\ \mathbf{K}\ \mathbf{K}$.

$$[x](f\,x) = \mathbf{S}([x]f)([x]x) = \mathbf{S}(\mathbf{K}\,f)\mathbf{I}$$

Check: applying it to N:

$$\mathbf{S}(\mathbf{K}\,f)\mathbf{I}\,N \rightsquigarrow \mathbf{K}\,f\,N(\mathbf{I}\,N) \rightsquigarrow f(\mathbf{I}\,N) \rightsquigarrow^* f\,N$$

Theorem 1.10 (Turing completeness)**.** Every function a Turing machine computes is computed by a closed λ-term, and every function a closed λ-term computes is computed by a Turing machine.

1.5 The simply typed λ-calculus

1.5.1 The Monster, in λ notation

The function that applies its argument to itself, applied to itself, runs forever:

$$\Omega = (\lambda x.\, x\, x)(\lambda x.\, x\, x) \rightsquigarrow (\lambda x.\, x\, x)(\lambda x.\, x\, x) \rightsquigarrow \ldots$$

1.5.2 Three rules

Types, contexts and judgements are those of the combinator checker: $\Gamma \vdash M : A$ says M has type A when its variables have types Γ gives them. Two rules are the checker's own. The third is the Deduction Theorem, adopted as a rule with the program written on.
(var):

$$\frac{}{\Gamma \vdash x : A}\ (\mathsf{var})\ \text{if } x : A \text{ is in } \Gamma$$

(app):

$$\frac{\Gamma \vdash M : A \to B \qquad \Gamma \vdash N : A}{\Gamma \vdash M\, N : B}\ (\text{app})$$

(abs):

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x.\, M : A \to B}\ (\text{abs})$$

Read (abs) as the checker reads a function definition: add the parameter to the context, check the body, give the function the arrow type.

1.5.3 Typings are trees now

Rule (abs) changed the context, so a typing is now a tree whose every node is a whole judgement, context included: natural deduction in sequent style.

Example 1.11 (function composition, the body)**.** The typing of a program we've written in one form or another: `\f -> \g -> \x -> f(g x)`. The body is checked in the context $f : B \to C, g : A \to B, x : A$:
Step 1:

$$\dfrac{\dfrac{}{f : B \to C,\ g : A \to B,\ x : A \vdash g : A \to B}\ \text{var} \qquad \dfrac{}{f : B \to C,\ g : A \to B,\ x : A \vdash x : A}\ \text{var}}{f : B \to C,\ g : A \to B,\ x : A \vdash g\, x : B}\ \text{app}$$

Step 2, using Step 1:

$$\dfrac{\dfrac{}{f : B \to C, g : A \to B,\ x : A \vdash g : A \to B}\ \text{var} \qquad (1)\ f : B \to C,\ g : A \to B,\ x : A \vdash g\, x : B}{f : B \to C,\ g : A \to B,\ x : A \vdash f\,(g\, x) : C}\ \text{app}$$

Three look-ups and two uses of (app), each matching an argument's type against the type a function expects.
Step 3: three abstractions

$$\dfrac{\dfrac{\dfrac{f : B \to C, g : A \to B, x : A \vdash f(g\,x) : C}{f : B \to C, g : A \to B \vdash \lambda x.f(g\,x) : A \to C}\,\text{abs}}{f : B \to C \vdash \lambda g.\lambda x.f(g\,x) : (A \to B) \to A \to C}\,\text{abs}}{\vdash \lambda f.\lambda g.\lambda x.f(g\,x) : (B \to C) \to (A \to B) \to A \to C}\,\text{abs}$$

- The top is a proof of C from $B \to C, A \to B$ and A, by use of modus ponens.
- The three (abs) bars discharge A, then $A \to B$, then $B \to C$.
- The first discharge alone gives $A \to C$ from $B \to C$ and $A \to B$: hypothetical syllogism, the rule that chains two implications.

$$\vdash (B \to C) \to (A \to B) \to A \to C$$

Hypothetical syllogism as a theorem, with nothing assumed. The compiler read our composition function as that proof.

Definition 1.12 (pairs, the empty type, sums, reading $(+E)$)**.** $A \times B$ is the pair type `(A, B)`; $\bot$ is the type with no constructor, `Void`, of which there are no values; $\neg A$ is $A \to \bot$.

$$\frac{\Gamma \vdash M : A \qquad \Gamma \vdash N : B}{\Gamma \vdash (M, N) : A \times B}\,(\times I)$$

$$\frac{\Gamma \vdash M : \bot}{\Gamma \vdash \text{absurd}\,M : C}\,(\bot E)$$

$$\frac{\Gamma \vdash M : A \times B}{\Gamma \vdash \text{fst}\,M : A}\,(\times E_1)$$

$$\frac{\Gamma \vdash M : A \times B}{\Gamma \vdash \text{snd}\,M : B}\,(E_2)$$

To build an $A + B$ we build both halves; to use one we take whichever half we need. Nothing builds a $\bot$, if we have one, we can conclude anything: `absurd :: Void -> a`.

$A + B$ is the sum type, `Either A B`:

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \text{Left}\,M : A + B}\,(+I_1)$$

$$\frac{\Gamma \vdash M : B}{\Gamma \vdash \text{Right}\,M : A + B}\,(+I_2)$$

$$\frac{\Gamma \vdash M : A \qquad \Gamma, x : A \vdash P : C \qquad \Gamma, y : B \vdash Q : C}{\Gamma \vdash \ \text{case}\ M\ \text{of}\ \{\mathit{Left}\,x \implies P | \mathit{Right}\,y \implies Q\} : C}\,(+E)$$

To build an $A + B$ we build one side and say which, to use one we argue by cases. On the page the pseudocode goes on one line, its two branches between braces:

$$\text{case}\ M\ \text{of}\ \{\mathit{Left}\,x \implies P | \mathit{Right}\,y \implies Q\} : C$$

- $M : A + B$: a tagged value to look at;
- $x : A$ in P only, $y : B$ in Q only: each branch has its own variable.
- $P : C$ and $Q : C$ only one runs, and what follows can't know which.

1.5.4 The central table

Connective	Type	Introduction (constructor)	Elimination (pattern match)
$A \to B$	$A \to B$, `A -> B`	$\lambda x.\, M$	application $M\, N$
$A \wedge B$	$A \times B$, `(A, B)`	(M, N)	*fst*, *snd*
$A \vee B$	$A + B$, `Either A B`	*Left* M, *Right* M	`case` with two branches
$\bot$	$\bot$, `Void`	—	*absurd*
$\neg A$	$A \to \bot$, `A -> Void`	$\lambda x.\, M$	application

The connectives of logic are the data types of a programming language; the introduction rules are their constructors, and the elimination rules are the pattern matches that take them apart.

Example 1.13 (swapping a pair)**.** $A \wedge B \to B \wedge A$ is a theorem, and here is the program that proves it: $\lambda p.(\mathit{snd}\, p, \mathit{fst}\, p) : A \times B \to B \times A$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{p : A \times B \vdash p : A \times B}\,\text{var}}{p : A \times B \vdash \mathit{snd}\, p : B}\,\times\text{E}_2 \quad \dfrac{\dfrac{\dfrac{}{p : A \times B \vdash p : A \times B}\,\text{var}}{p : A \times B \vdash \mathit{fst}\, p : A}\,\times\text{E}_1}{p : A \times B \vdash (\mathit{snd}\, p,\ \mathit{fst}\, p) : B \times A}\,\times\text{I}}{\vdash \lambda p.\,(\mathit{snd}\, p,\ \mathit{fst}\, p) : A \times B \to B \times A}\,\text{abs}$$

Example 1.14 (contraposition)**.** $(A \to B) \to (\neg A \to \neg B)$: the program is $\lambda f.\lambda k.\lambda a.k(f\, a)$. Its body, typed in the context $f : A \to B, k : B \to \bot, a : A$:
Step 1:

$$\dfrac{\dfrac{}{f : A \to B,\ k : B \to \bot,\ a : A \vdash f : A \to B}\,\text{var} \quad \dfrac{}{f : A \to B,\ k : B \to \bot,\ a : A \vdash a : A}\,\text{var}}{f : A \to B,\ k : B \to \bot,\ a : A \vdash f\, a : B}\,\text{app}$$

Step 2, using Step 1:

$$\dfrac{\dfrac{}{f : A \to B,\ k : B \to \bot,\ a : A \vdash k : B \to \bot}\,\text{var} \quad (1)\ f : A \to B,\ k : B \to \bot,\ a : A \vdash f\, a : B}{f : A \to B,\ k : B \to \bot,\ a : A \vdash k\,(f\, a) : \bot}\,\text{app}$$

Three uses of (abs) give $(A \to B) \to (B \to \bot) \to A \to \bot$.

Example 1.15 (swapping a union)**.** $A \vee B \to B \vee A$:

$$\lambda s.\ \textbf{case}\ s\ \textbf{of}\ \{\mathit{Left}\, a \implies \mathit{Right}\, a | \mathit{Right}\, b \implies \mathit{Left}\, b\}$$

The two branch variables a, b are the two leaves $[p]^2$ and $[q]^3$ of the tree we drew for this formula, with A, B for its p, q each in scope only in its own branch.

Theorem 1.16 (Curry-Howard)**.** In one line: $\Gamma \vdash M : A$ for some term M if and only if $|\Gamma| \vdash A$.

1. **(Erasure)** Take a typing of $\mathit{Gamma} \vdash M : A$, delete every term, and write $\wedge$ for $\times$, *lor* for $+$. Then (var) becomes a hypothesis, (app) an $\to E$ bar, (abs) an $\to I$ bar, and so on. The result is a derivation of A with open hypotheses among $|\Gamma|$, the types declared in Γ.

2. **(Decoration)** Take a derivation of A, write $\times$ for $\wedge$, $+$ for $\vee$, choose a variable for each hypothesis, and write terms upwards from the leaves: a variable at each leaf, an application at each $\to E$, a λ at each $\to I$, a constructor or pattern match at each other bar. The result is a typing of $\Gamma \vdash M : A$, with Γ declaring open hypotheses.

why:
Put the typing rules beside the clauses that define derivations. Rule for clause, each derivation clause is the typing rule with the terms removed and the two symbols renamed: premises, conclusions, and what each takes out of the context or out of OH are otherwise identical.

$$\begin{aligned} 2\mathrm{FV}(\lambda x.M) &= \mathrm{FV}(M)\backslash\{x\} && (\text{a }\lambda \text{ binds its variable} \\ \mathrm{OH} &= \mathrm{OH}(\mathcal{D})\backslash\{\mathrm{A}\} && \text{an } \to I \text{ bar discharges its formula} \end{aligned}$$

The theorem says that a derivation in Gentzen's system and a typing of a program with parameters, pairs, unions and `case` are the same **same tree with a column added**; and that the rule which puts a parameter in scope and takes it out again is, column removed, the rule for supposing and discharging.

1.6 Normalisation and consistency

1.6.1 Detours

A detour, in Gentzen's word, is an introduction met at once by its own elimination: prove $A \to B$ by $\to I$, then at once use it on a proof ε of A. The proof is shorter without the pair:

$$\dfrac{\dfrac{\begin{array}{c}[A]\\ \mathcal{D}\\ B\end{array}}{A \to B}\,{\to}\mathrm{I} \qquad \begin{array}{c}\mathcal{E}\\ A\end{array}}{B}\,{\to}\mathrm{E} \qquad \text{becomes} \qquad \begin{array}{c}\mathcal{E}\\ A\\ \mathcal{D}\\ B\end{array}$$

A copy of ε goes in place of every leaf $[A]$ of $\mathcal{D}$, and the two bars go.

Definition 1.17 (β-reduction, for the whole language)**.** A step rewrites an introduction met by its elimination, anywhere inside a term; by cases on the shape of M:

$$\begin{cases} (\lambda x.\, P)\, Q \rightsquigarrow P[Q/x], \\ \mathit{fst}\,(P, Q) \rightsquigarrow P \quad \text{and} \quad \mathit{snd}\,(P, Q) \rightsquigarrow Q, \\ \textbf{case}\ (\mathit{Left}\ R)\ \textbf{of}\ \{\ \mathit{Left}\, x \Rightarrow P \mid \mathit{Right}\, y \Rightarrow Q\ \} \rightsquigarrow P[R/x], \\ \textbf{case}\ (\mathit{Right}\ R)\ \textbf{of}\ \{\ \mathit{Left}\, x \Rightarrow P \mid \mathit{Right}\, y \Rightarrow Q\ \} \rightsquigarrow Q[R/y], \end{cases}$$

and a step inside a term is a step of the term. There is no line for *absurd*: $\bot$ has no introduction for it to meet. A term in which no step can be taken is in normal form; through Curry-Howard, a derivation with no detour is a normal derivation.

Theorem 1.18 (subject reduction)**.** If $\Gamma \vdash M : A$ and $M \rightsquigarrow N$, then $\Gamma \vdash N : A$. *Proof sketch.* One new ingredient, a substitution lemma: if $\Gamma, x : A \vdash M : B$ and $\Gamma \vdash N : A$, then $\Gamma \vdash M[N/x] : B$. Go down the typing of M and replace every (var) line for x by the typing of N. ON the proof side is transitivity.
Then one case per step. For $\beta\,(\lambda x.M)N : B$ came by (app) from $\lambda x.M : A \to B$ and $N : A$; the first came by (abs) from $\Gamma, x : A \vdash M : B$; the lemma gives $M[N/x] : B$

Theorem 1.19 (normalisation)**.** Every typed term runs to a normal form. Equivalently: every derivation can be rewritten, by removing detours, into a normal derivation of the same formula from the same open hypotheses.

Definition 1.20 (normal and neutral terms)**.** A typed term is neutral if it is a variable, or an elimination applied to a neutral term:

$$N\,M, \qquad \mathit{fst}N, \qquad \mathit{snd}N, \qquad \text{case } N \text{ of } \{...\}, \qquad \mathit{absurd}N$$

with N neutral and the other parts in normal form.
A form is in normal form exactly when it is an introduction form, $\lambda x.M$, (M, N), $\mathit{Left}M$ or $\mathit{Right}M$, whose parts are in normal form, or is neutral. Why "exactly when": the types do work.

- Take a normal-form application $M\,N$. Its M is in normal form and has an arrow type, and it is not a λ, or $M\,N$ would be a redex.
- By induction, M is an introduction form or neutral. The only introduction form with an arrow type is a λ. So M is neutral, and then so is $M\,N$.
- The same argument handles a projection, whose argument cannot be a pair, and a case, whose argument cannot be a tag.
- The argument of an *absurd* has type $\bot$, which has no introduction form at all, so it too must be neutral.

Lemma 1.21 (shape of closed normal forms)**.** A closed type term in normal form is an introduction form. Consequently there is no closed term in normal form whose type is a letter P, and none whose type is $\bot$.

Theorem 1.22 (consistency)**.** There is no closed term type $\bot \nvdash \bot$. More generally, for every letter p, $\nvdash p$

Corollary 1.23 (disjunction property)**.** If $\vdash A \vee B$, then $\vdash A$ or $\vdash B$

1.7 Fitch proofs

Definition 1.24 (Fitch proof, the boxes)**.** A Fitch proof φ from Γ is a box with head Γ whose last entry is the line φ, in which every line φ_k is justified in one of these ways, "earlier" meaning "in scope at line k":

1. (*hyp*) φ_k is in Γ, or is the supposition of a subbox that contains line k;
2. ($\to E$) there are earlier lines φ_i and φ_j such that φ_i is $\varphi_j \to \varphi_k$;
3. ($\to I$) φ_k is $\psi \to \chi$, and there is an earlier subbox with supposition ψ whose last line is χ;
4. ($\wedge I$) φ_k is $\psi \wedge \chi$, and ψ and χ are earlier lines;
5. ($\wedge E_1, \wedge E_2$) some earlier line is $\varphi_k \wedge \chi$, or $\psi \wedge \varphi_k$;
6. ($\vee I_1, \vee I_2$) φ_k is $\psi \vee \chi$, and ψ, or χ, is an earlier line;
7. ($\vee E$) some earlier line is $\psi \vee \chi$, and there are two earlier subboxes, with suppositions ψ and χ, each with last line φ_k;
8. ($\bot E$) $\bot$ is an earlier line.

No other kind of line is allowed.

Theorem 1.25 (trees and boxes)**.** For every set of formulas Γ and every formula φ, the following are equivalent:

1. $\Gamma \vdash_N \varphi$: some tree derivation has conclusion φ and all its open hypotheses in Γ;
2. φ has a Fitch proof of Γ.

The translations both ways go step by step: each bar of a tree becomes a line justified by the same rule; each line of a Fitch proof not justified by (hyp) becomes a bar of the same rule.